Parallel Monte Carlo Risk Engine

CSC209 Systems Programming — Project Documentation

---

1. Project Overview

This project implements a **parallel Monte Carlo Value-at-Risk (VaR) engine** in C. It belongs to **Category 1: Multi-Process Systems**, using `fork()`-based worker processes connected to a parent controller via UNIX pipes. The project is non-interactive: all inputs arrive as command-line arguments and all output is printed to stdout/stderr.

What the program computes. A *European call option* gives its holder the right to buy a stock at a fixed price (strike K) on a future date. Its market value changes every day as the stock price moves. *Value at Risk* (VaR) answers: "on a bad day, how much could I lose?" — specifically, the loss threshold that is exceeded on only 5% (or 1%) of days. VaR is a standard risk metric required by Basel III regulation at banks and hedge funds. This engine estimates 1-day empirical VaR for a single long call position.

How it works. The **controller process** (parent) parses arguments, forks N worker processes, and distributes simulation work. Each **worker process** independently simulates its share of Monte Carlo paths: it models 1-day stock moves using Geometric Brownian Motion (GBM), re-prices the option via the Black-Scholes formula after each move, and records the resulting profit-and-loss (P&L;). Workers return P&L; arrays to the controller via pipes. The controller aggregates results, sorts the full P&L; distribution, and computes the empirical 95% and 99% 1-day VaR, mean P&L;, and standard deviation.

Input: seven positional command-line arguments plus one optional seed:

```
./risk_engine <num_workers> <num_sims> <S0> <K> <r> <sigma> <T_years> [seed]

Example:  ./risk_engine 4 100000 100 100 0.05 0.2 0.25 42
```

Output: a formatted block printed to stdout with model parameters, option price at t=0, mean P&L;, standard deviation, 95% VaR, 99% VaR, and wall-clock runtime. Validation errors go to stderr.

Key features:

- N ≥ 3 parallel worker processes running Monte Carlo simulations independently
- Binary pipe IPC with a defined two-message protocol (see Section 4)
- `select()`-based result collection: drains the fastest worker first
- Reproducible runs via a configurable base PRNG seed (derived per-worker with Knuth hash)
- Full error handling: every syscall checked; zombie prevention via `waitpid()`

---

2. Build Instructions

Requirements: `gcc` with C11 support, GNU `make`, a POSIX-compliant OS (Linux or macOS). No external libraries beyond libc and libm are needed. The flag `-D_DEFAULT_SOURCE` is required to expose `rand_r()`, `clock_gettime()`, and math constants (`M_PI`, `M_SQRT2`) on Linux with `-std=c11`.

```
cd risk_engine
make              # compile; produces ./risk_engine binary
make clean        # remove .o, .d files, and the binary
make run          # convenience: build + run with default parameters
```

Running the program:

```
./risk_engine <num_workers> <num_sims> <S0> <K> <r> <sigma> <T_years> [seed]

# At-the-money call, 4 workers, 100 000 paths, default seed
./risk_engine 4 100000 100 100 0.05 0.2 0.25

# Same parameters, explicit seed for a different sample
./risk_engine 4 100000 100 100 0.05 0.2 0.25 42
```

Constraints enforced before forking: `num_workers` ≥ 3; `num_sims` > 0; `S0`, `K`, `sigma` > 0; `T_years` > 1/252 (at least one trading day). Each violated constraint produces a distinct error message to stderr before the program exits.

---

3. Architecture Diagram

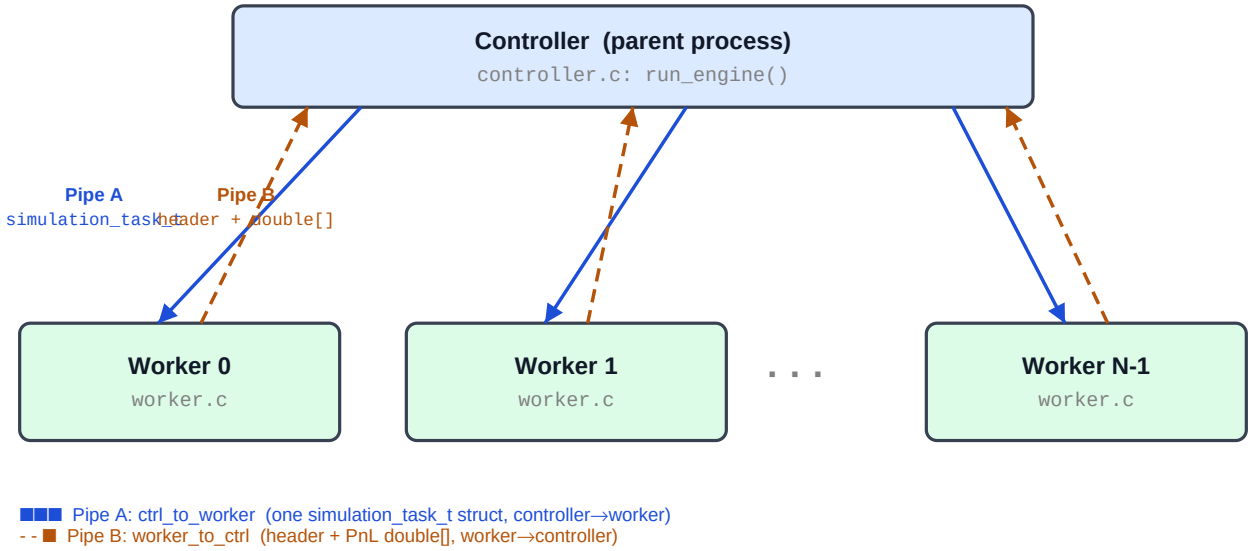

Figure 1. Process and pipe topology. Solid blue arrows = Pipe A (controller → worker). Dashed amber arrows = Pipe B (worker → controller). Workers share no memory; all communication is through pipes.

All source files involved: `main.c` (entry point, timing, output), `controller.c` (orchestration, `run_engine()`), `worker.c` (`run_worker()`), `mc.c` (GBM simulation), `option_pricing.c` (Black-Scholes), `stats.c` (empirical VaR), `util.c` (I/O helpers).

---

4. Communication Protocol

All IPC uses raw binary structs defined in `protocol.h`. Reads and writes go through `util.c`: `read_all()` (lines 19–38) and `write_all()` (lines 40–54), which loop until all bytes are transferred and retry on EINTR, handling short reads/writes transparently.

Message 1 — Task Dispatch (Pipe A: controller → worker)

Sender: controller (`controller.c`, lines 83–101). **Receiver:** each worker (`worker.c`, line 13).

Encoding: one fixed-size struct, written in a single `write_all()` call:

```
typedef struct {
    int          worker_id;   /* 4 bytes — which worker this is       */
    int          num_sims;    /* 4 bytes — paths assigned             */
    uint32_t     seed;        /* 4 bytes — per-worker PRNG seed       */
    model_params_t params;    /* 48 bytes — S0,K,r,sigma,T,dt         */
} simulation_task_t;          /* ~60 bytes total                      */
```

Semantics: The receiver calls `read_all(task_read_fd, &task;, sizeof(task))` exactly once (`worker.c:13`). The byte count is known statically, so no framing header is needed. After reading, the worker closes `task_read_fd` (`worker.c:17`) — it will not read again.

EOF signalling: After writing the task, the controller closes its write end of Pipe A (`controller.c:101`). This ensures the worker sees EOF from `read_all()` if the struct arrives partially or the controller dies unexpectedly. On send failure, remaining write ends are still closed so workers are not left blocking on an empty pipe.

Message 2 — Result Delivery (Pipe B: worker → controller)

Sender: worker (`worker.c`, lines 24–40). **Receiver:** controller (`controller.c`, lines 164–196).

Encoding: two sequential writes:

```
/* Write 1: fixed-size framing header (8 bytes) */
typedef struct {
    int worker_id;   /* echoed back for protocol validation */
    int num_sims;    /* declares how many doubles follow   */
} worker_result_header_t;

/* Write 2: variable-length payload */
write_all(result_write_fd, pnl, num_sims * sizeof(double));
```

Semantics: The controller first reads the 8-byte header (`controller.c:164–172`) to determine payload size, then reads exactly `header.num_sims × 8` bytes into the pre-computed slice of the aggregated buffer at `all_pnl + offset_for[i]` (`controller.c:187–192`). The offset for worker `i` is stored in `offset_for[i]` (`controller.c:113–119`), computed before any results arrive, because workers may return results out of index order.

Invariant checked by receiver: after reading the header, the controller validates that `header.worker_id == i` and `header.num_sims == sims_per_worker[i]` (`controller.c:174–185`). A mismatch closes the fd immediately without reading the payload, preventing a buffer overrun or an infinite blocking read.

---

5. Concurrency Model

This project uses **Category 1 concurrency**: one parent process forks N independent child processes. Workers share no memory after `fork()`; all synchronization is implicit in the pipe I/O. The PRNG state is fully local to each worker process (via `rand_r()`), so no locking is needed.

When the parent forks

All 2N pipes are created before any `fork()` call (`controller.c:39–42`), so every child inherits a full set of file descriptors. Immediately after `fork()`, the child closes every fd it does not own (`controller.c:52–62`): it keeps only `ctrl_to_worker[i][0]` (read task) and `worker_to_ctrl[i][1]` (write results). Leaving extra write-end copies open in the child would prevent the controller from ever seeing EOF on those pipes — a classic pipe bug. The fork loop runs N times (`controller.c:44–67`); each child calls `run_worker()` immediately and does not wait for siblings.

How workers know when to start

There is no explicit "ready" handshake. The controller sends tasks immediately after forking all workers (`controller.c:83–101`). Because each task struct is tiny (<100 bytes) and fits in the kernel pipe buffer, `write_all()` returns before the worker necessarily starts reading. Workers begin simulation as soon as `read_all()` returns on the task pipe (`worker.c:13`). All N workers run their Monte Carlo loops concurrently on separate cores with no inter-worker communication.

The select() result-collection loop

After sending all tasks, the controller enters the collection loop (`controller.c:132–198`). `select()` is used instead of sequential reads to avoid head-of-line blocking: if worker 3 finishes while the controller is waiting for worker 0, worker 3's results would sit unused in the pipe buffer. `select()` blocks until *any* pending worker pipe becomes readable, eliminating this inefficiency.

The fd set. In each loop iteration, the controller calls `FD_ZERO()` and adds `worker_to_ctrl[i][0]` to `readfds` for every i where `finished[i] == 0` (`controller.c:133–143`). `maxfd` tracks the highest fd number, which `select()` requires as its first argument.

Draining all ready workers per round. After `select()` returns, the controller iterates over all workers and processes every fd flagged in the ready set (`controller.c:160–197`). For each ready fd it reads the 8-byte header, validates it, then calls `read_all()` for the full P&L; payload. Processing all ready fds per iteration avoids an unnecessary `select()` syscall when multiple workers finish simultaneously. `EINTR` on `select()` is retried via `continue` (`controller.c:147`).

Fixed-slice offsets. Because workers may return results in any order, each worker's P&L; values must be written into a predetermined, non-overlapping region of the shared buffer. `offset_for[i]` (`controller.c:113–119`) maps worker index to buffer position and is computed once before any collection begins.

Collecting child processes

After the `select()` loop, all pipe I/O is complete. The controller calls `waitpid(pids[i], &status;, 0)` for each worker (`controller.c:216–223`). `waitpid()` blocks until each child exits and reaps its process-table entry, preventing zombie processes. The exit status is checked with `WIFEXITED()` and `WEXITSTATUS()`; an abnormal exit is reported to stderr but does not abort the controller, as results were already collected or marked as failed.

6. Error Handling and Robustness

Bad Behaviour 1: Worker crashes mid-write (partial P&L; payload)

Scenario: A worker is killed (e.g., out-of-memory or SIGKILL) after writing the result header but before finishing the P&L; array. The controller reads a valid header but then gets an unexpected EOF when it tries to read the payload.

Handling: `read_all()` (`util.c:30–34`) detects EOF when `read()` returns 0 before `total == count`:

```
30  if (n == 0) {
31      fprintf(stderr, "read_all: unexpected EOF (%zu/%zu bytes)\n",
32              total, count);
33      return -1;
34  }
```

The caller at `controller.c:188–192` checks the return value, sets `collect_ok = 0`, closes the fd, and marks this worker as finished. The loop continues draining remaining workers. After all workers are processed, `run_engine()` returns -1, and `main()` prints "Fatal: engine run failed." and exits with `EXIT_FAILURE`.

Bad Behaviour 2: Worker writes to a broken pipe

Scenario: The controller exits unexpectedly while a worker is still writing its P&L; array. The read end of Pipe B is now closed.

Handling: The kernel delivers SIGPIPE (or returns EPIPE if the signal is ignored). `write_all()` (`util.c:45–49`) catches the -1 return, calls `perror("write")`, and returns -1. The caller in `worker.c:36–40` checks the return value, frees the P&L; buffer, and calls `exit(EXIT_FAILURE)`:

```
36  if (write_all(result_write_fd, pnl, pnl_bytes) != 0) {
37      fprintf(stderr, "worker %d: failed to write PnL\n", task.worker_id);
38      free(pnl);
39      exit(EXIT_FAILURE);
40  }
```

Bad Behaviour 3: Malformed result header (protocol mismatch)

Scenario: Due to a bug or memory corruption a worker sends a header with an incorrect worker_id or num_sims. Trusting these values would cause the controller to read the wrong number of bytes, corrupting the aggregated buffer or blocking indefinitely.

Handling: After reading the header, the controller validates both fields (`controller.c:174–185`):

```
174  if (header.worker_id != i || header.num_sims != sims_per_worker[i]) {
175      fprintf(stderr,
176              "controller: protocol mismatch from worker %d "\n            "(expected id=%d sims=%d, got id=
177      collect_ok = 0;
178      close(worker_to_ctrl[i][0]);
179      finished[i] = 1;
180      workers_done++;
181      continue;
182  }
```

A mismatch causes the controller to close the fd without reading the payload, preventing a buffer overrun or a hang on an incorrect byte count.

Bad Behaviour 4: Memory allocation failure

Scenario: A `malloc()` call fails (e.g., insufficient memory for a large P&L; buffer). An unchecked NULL return would cause a segfault on the next dereference.

Handling: Every heap allocation goes through `xmalloc()` (`util.c:13–17`), which checks the return value and calls `die()` on failure:

```
 13  void *xmalloc(size_t size) {
 14      void *ptr = malloc(size);
 15      if (!ptr) die("malloc");   /* perror + exit(EXIT_FAILURE) */
 16      return ptr;
 17  }
```

This guarantees a clean exit with a diagnostic message rather than a silent crash. `xmalloc()` is used for all heap allocations in the program: pipe arrays, PID array, P&L; buffer, and per-worker tracking arrays.